

Word embeddings

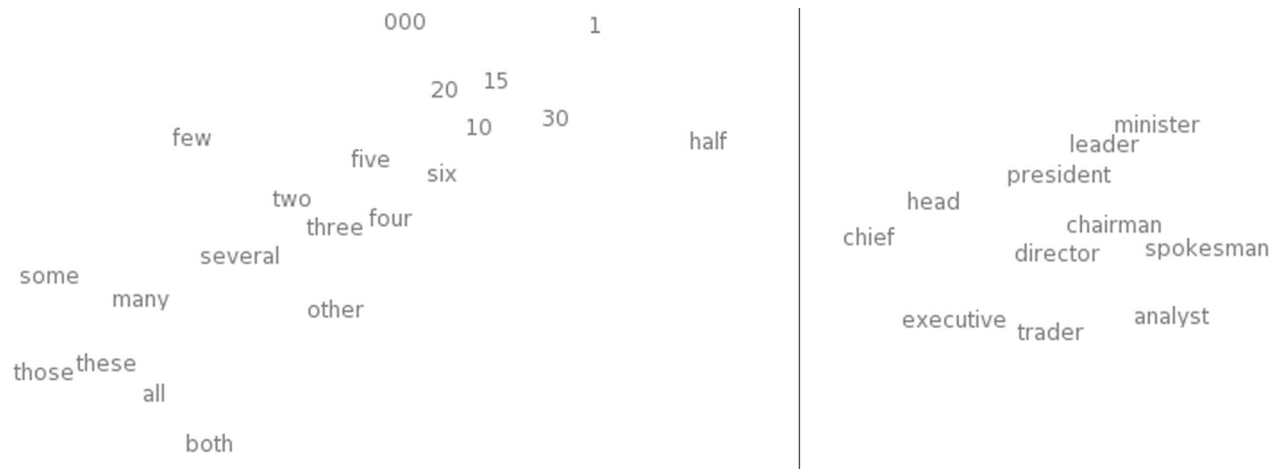
- Text processing with current NNs requires encoding into vectors.
- One-hot encoding:
 - N words encoded by length N vectors.
 - A word gets a vector with exactly one entry = 1, others 0.
 - Very space inefficient, no natural language structure.
- Bag of words:
 - Collection of words (along with number of occurrences).
 - No word order, no natural language structure.

Word embeddings: idea

- **Idea:** learn an embedding from words into vectors

Word embeddings: similarity

- Hope to have similar words nearby



Word embeddings: relationships

- Hope to preserve some language structure (relationships between words).

Relationship	Example 1	Example 2	Example 3
France - Paris	Italy: Rome	Japan: Tokyo	Florida: Tallahassee
big - bigger	small: larger	cold: colder	quick: quicker
Miami - Florida	Baltimore: Maryland	Dallas: Texas	Kona: Hawaii
Einstein - scientist	Messi: midfielder	Mozart: violinist	Picasso: painter
Sarkozy - France	Berlusconi: Italy	Merkel: Germany	Koizumi: Japan
copper - Cu	zinc: Zn	gold: Au	uranium: plutonium
Berlusconi - Silvio	Sarkozy: Nicolas	Putin: Medvedev	Obama: Barack
Microsoft - Windows	Google: Android	IBM: Linux	Apple: iPhone
Microsoft - Ballmer	Google: Yahoo	IBM: McNealy	Apple: Jobs
Japan - sushi	Germany: bratwurst	France: tapas	USA: pizza

Word embeddings: relationships

- **Capturing Relationships**

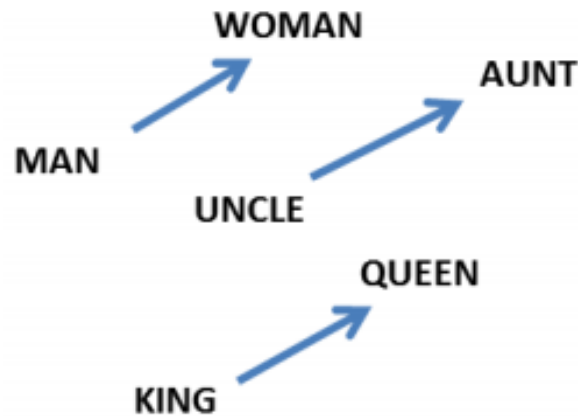
- The table in your slide illustrates different types of relationships that embeddings can capture:
- **Geographical Entities to Capitals:** Like France to Paris, Italy to Rome. This shows that embeddings can learn geographical hierarchies.
- **Comparative and Superlative Forms:** Such as big to bigger, cold to colder, illustrating morphological awareness.
- **Part-Whole Relationships:** Miami to Florida, where embeddings capture locality within a larger context.
- **Professional Titles or Roles:** Einstein to scientist, Messi to midfielder, connecting individuals to their professions or roles.
- **Entity and Its Creators or Notable Figures:** Microsoft to Ballmer (Steve Ballmer was a CEO of Microsoft), showing how entities are associated with key people.
- **Chemical Elements and Their Symbols:** Copper to Cu, gold to Au, indicating that embeddings can capture symbolic representations.
- **Cultural Associations:** Japan to sushi, Germany to bratwurst, linking countries with iconic foods or items.

Word embeddings: properties

- Need to have a function $W(\text{word})$ that returns a vector encoding that word.
- Similarity of words corresponds to nearby vectors.
 - Director – chairman, scratched – scraped
- Relationships between words correspond to difference between vectors.
 - Big – bigger, small – smaller

Word embeddings: properties

- Relationships between words correspond to difference between vectors.



$$W(\text{"woman"}) - W(\text{"man"}) \simeq W(\text{"aunt"}) - W(\text{"uncle"})$$

$$W(\text{"woman"}) - W(\text{"man"}) \simeq W(\text{"queen"}) - W(\text{"king"})$$

Word embeddings: questions

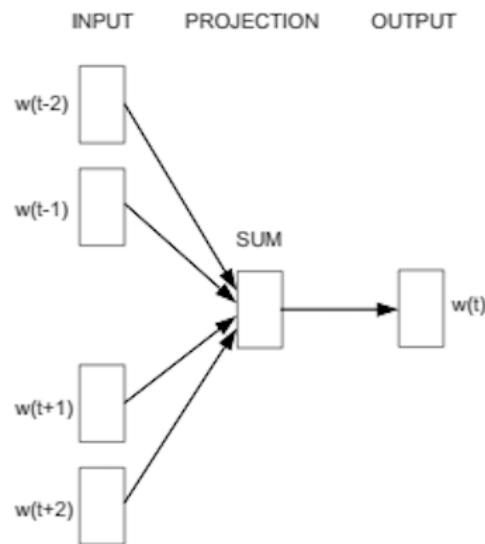
- How big should the embedding space be?
 - Trade-offs like any other machine learning problem – greater capacity versus efficiency and overfitting.
- How do we find W ?
 - Often as part of a prediction or classification task involving neighboring words.

Word embeddings (continued)

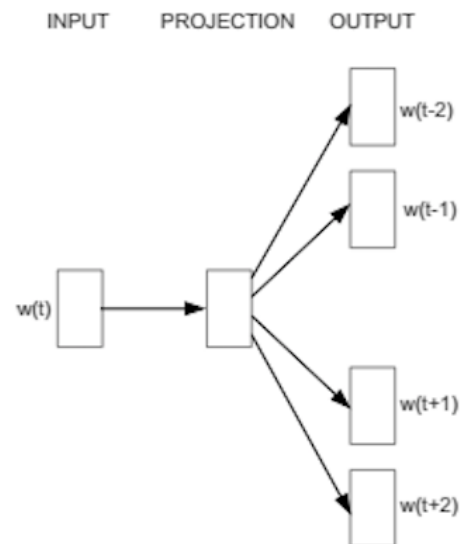
- **Idea:** learn an embedding from words into vectors
- Need to have a function $W(\text{word})$ that returns a vector encoding that word.

word2vec

- Predict words using context
- Two versions: CBOW (continuous bag of words) and Skip-gram



CBOW

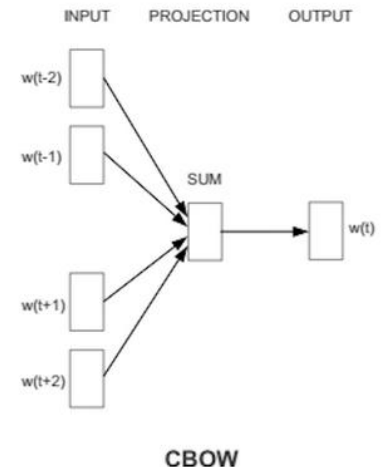


Skip-gram

DANIYAL ADEEB

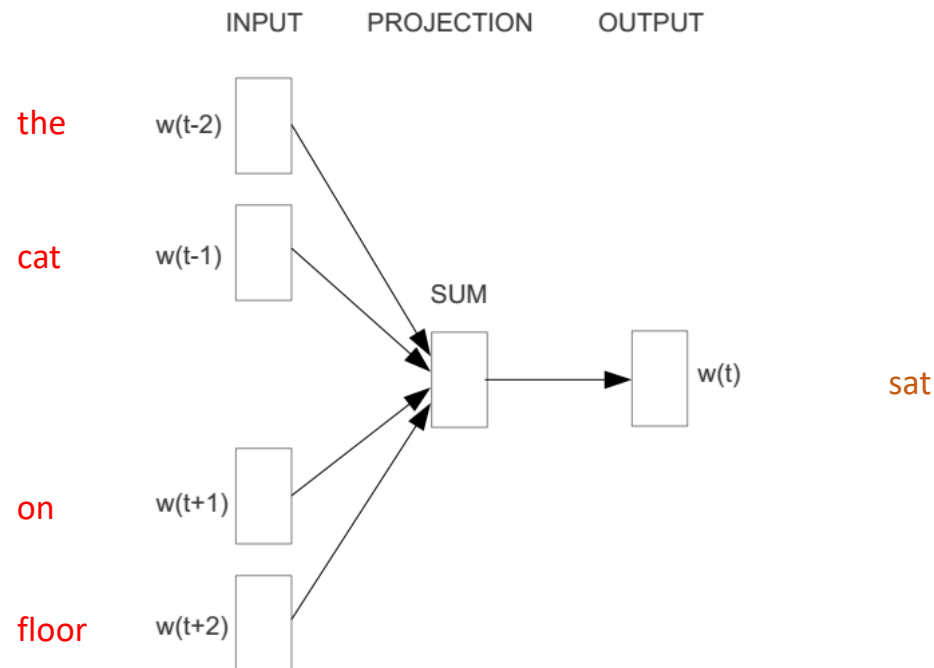
CBOW

- Bag of words
 - Gets rid of word order. Used in discrete case using counts of words that appear.
- CBOW
 - Takes vector embeddings of n words before target and n words after and adds them (as vectors).
 - Also removes word order, but the vector sum is meaningful enough to deduce missing word.



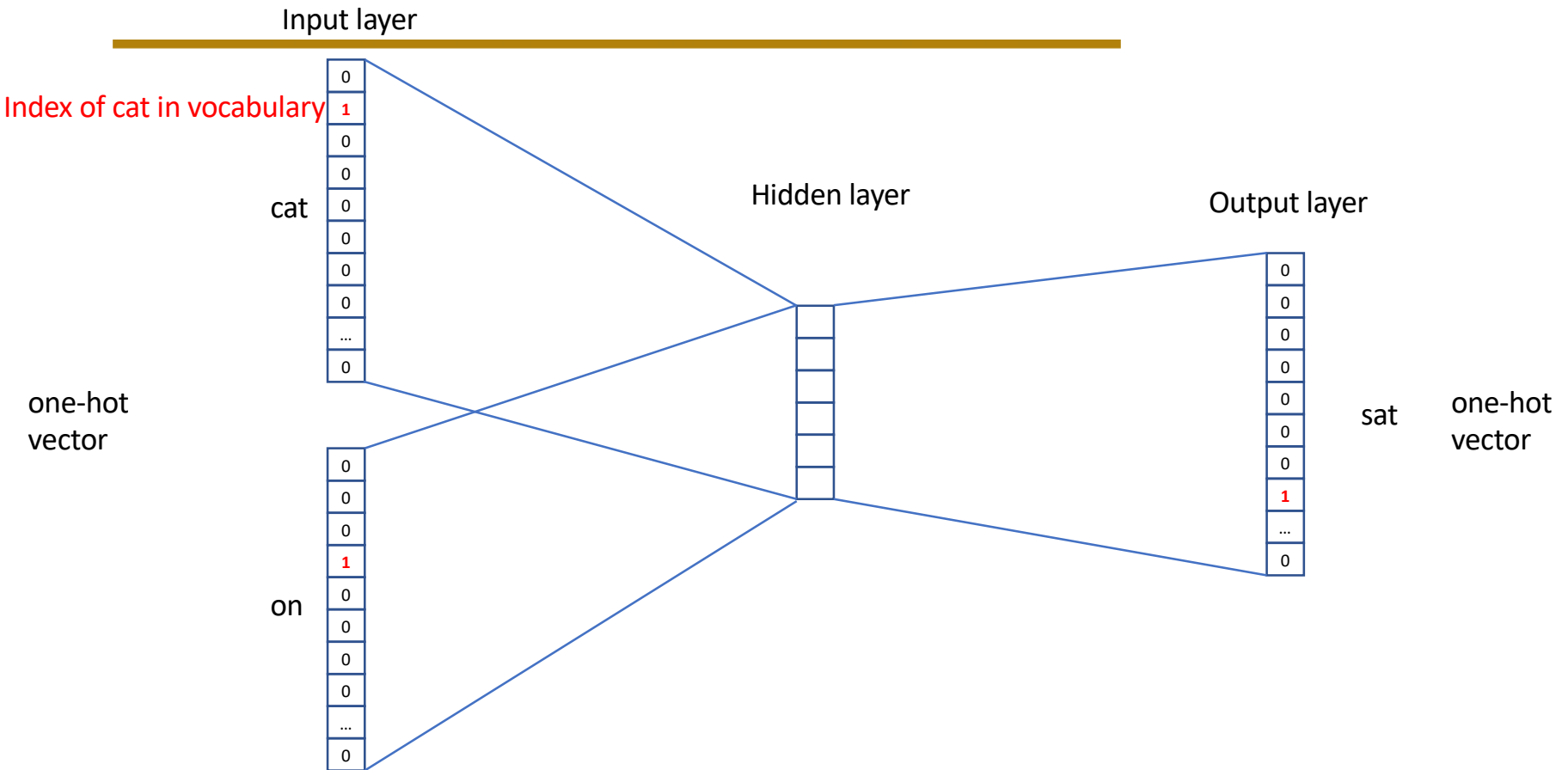
Word2vec – Continuous Bag of Word

- E.g. “The cat sat on floor”
 - Window size = 2

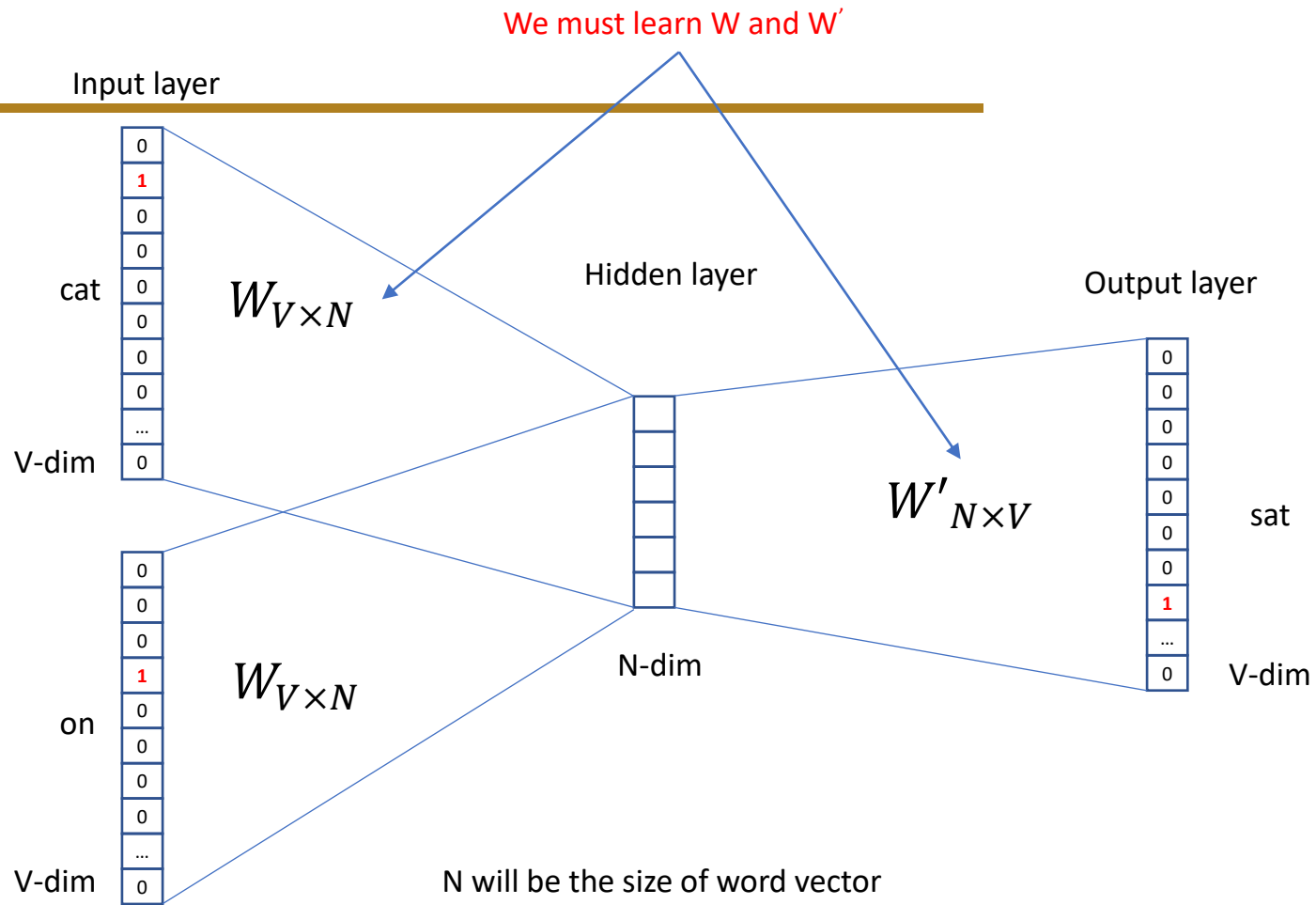


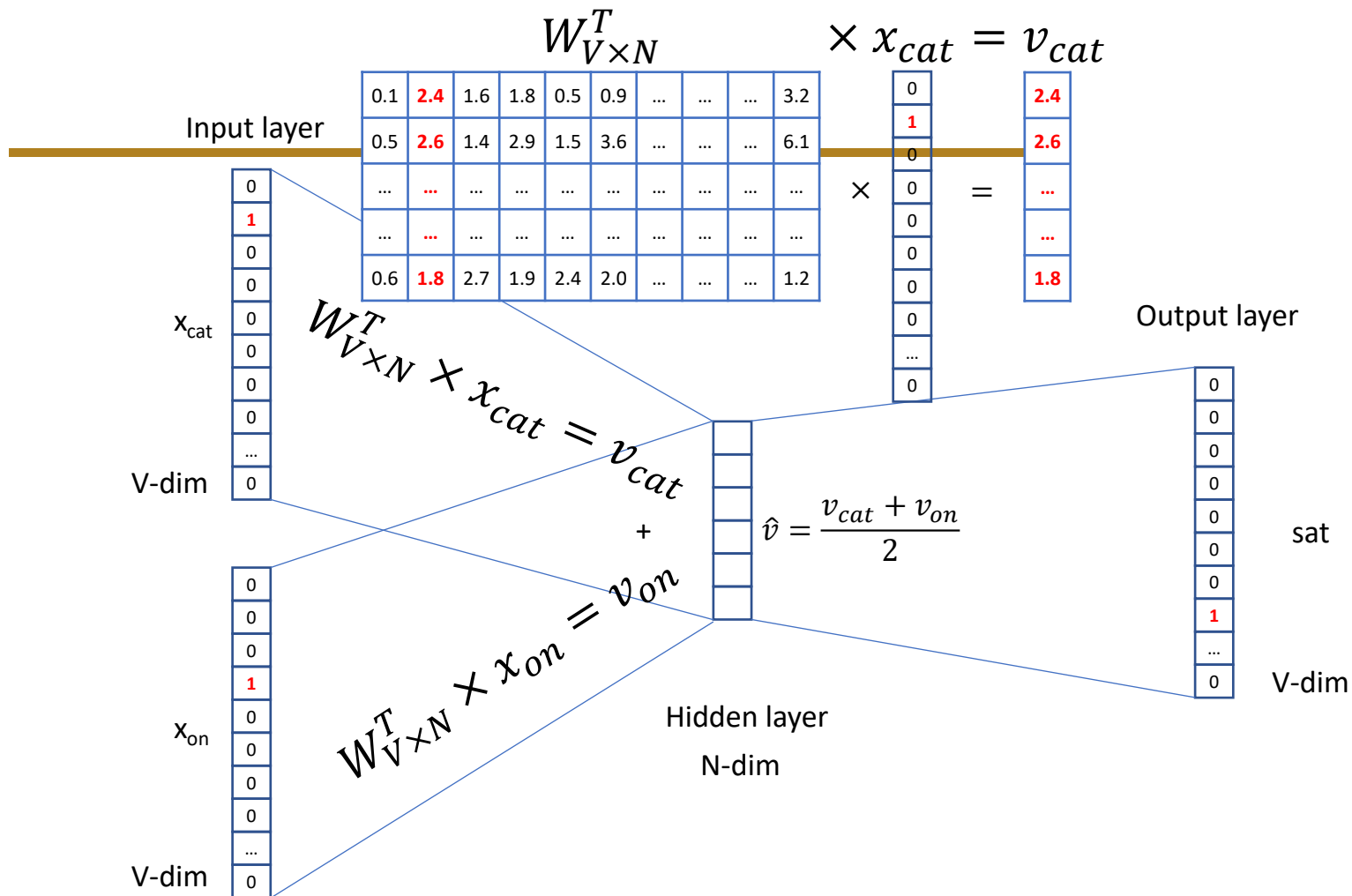
Word2vec – Continuous Bag of Word

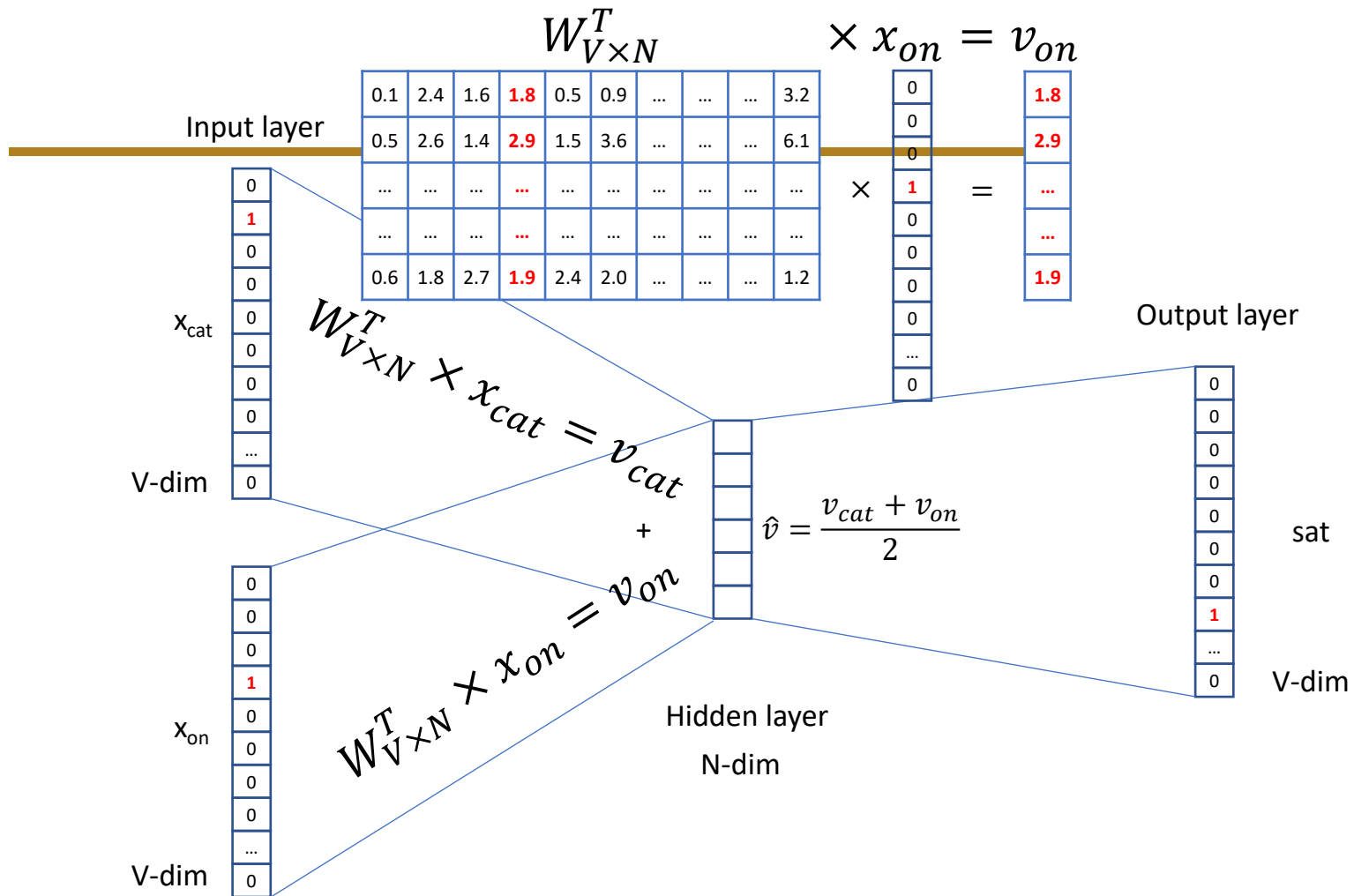
– Window Size = 1

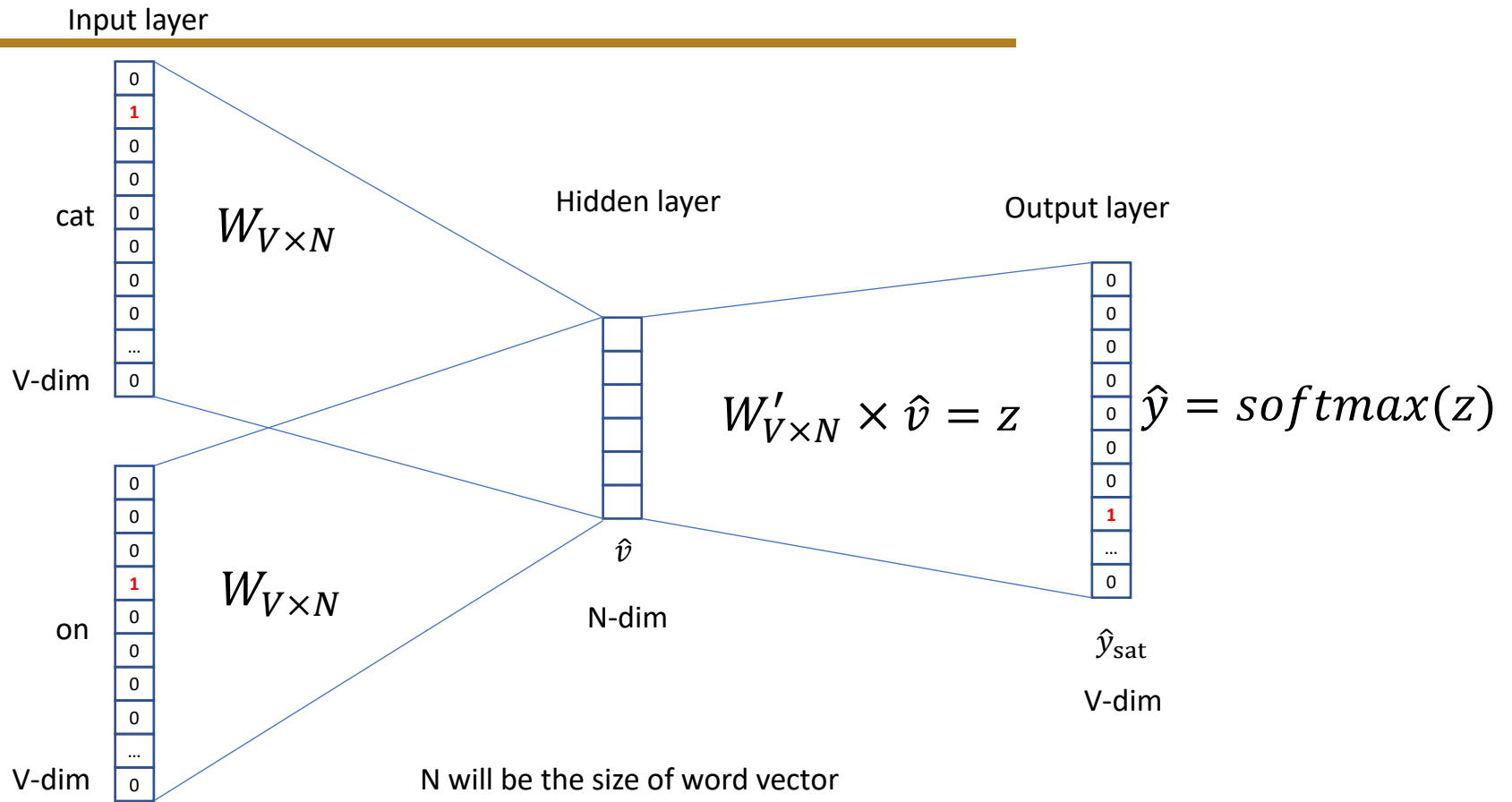


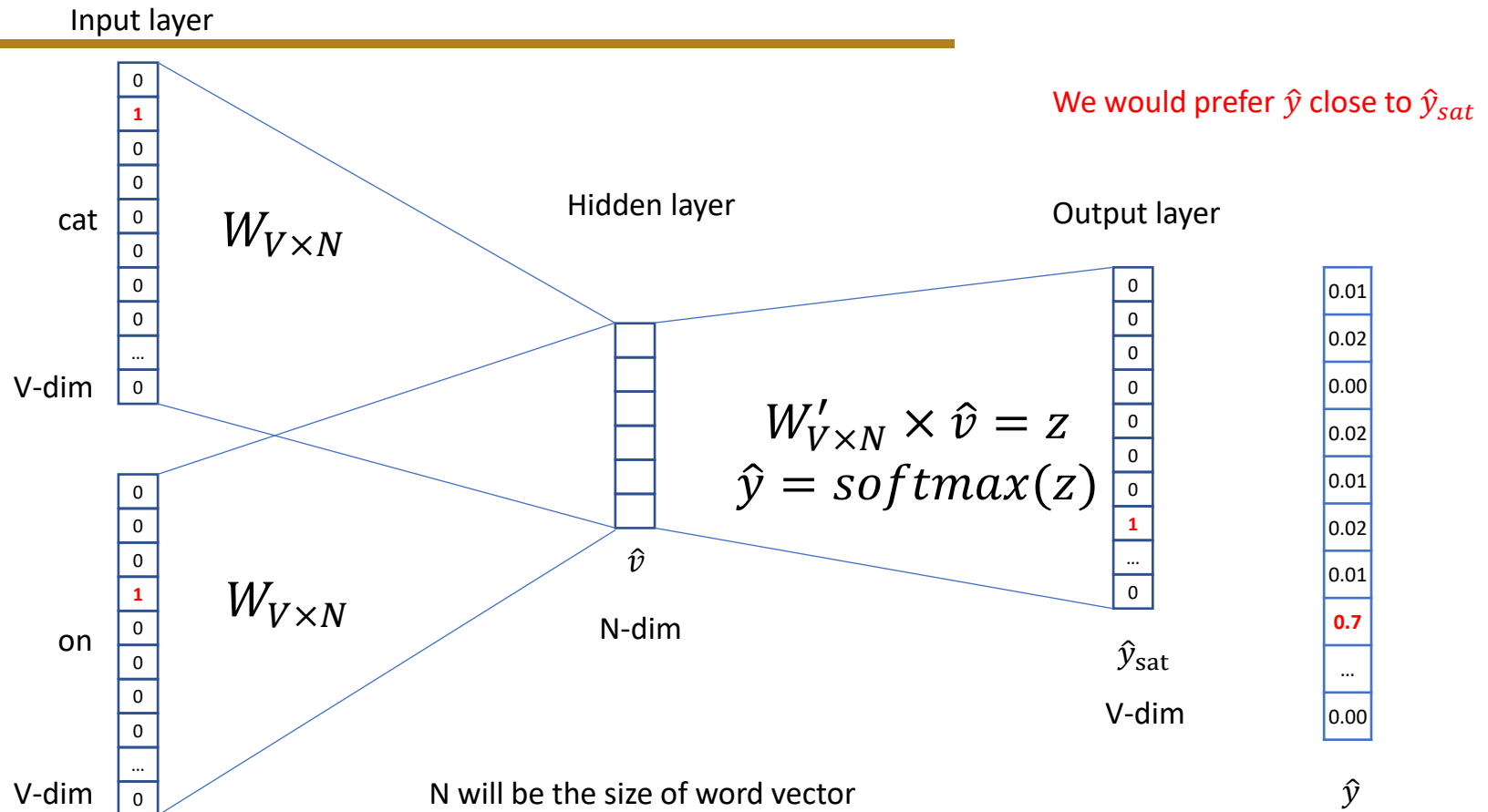
DANIYAL ADEEB

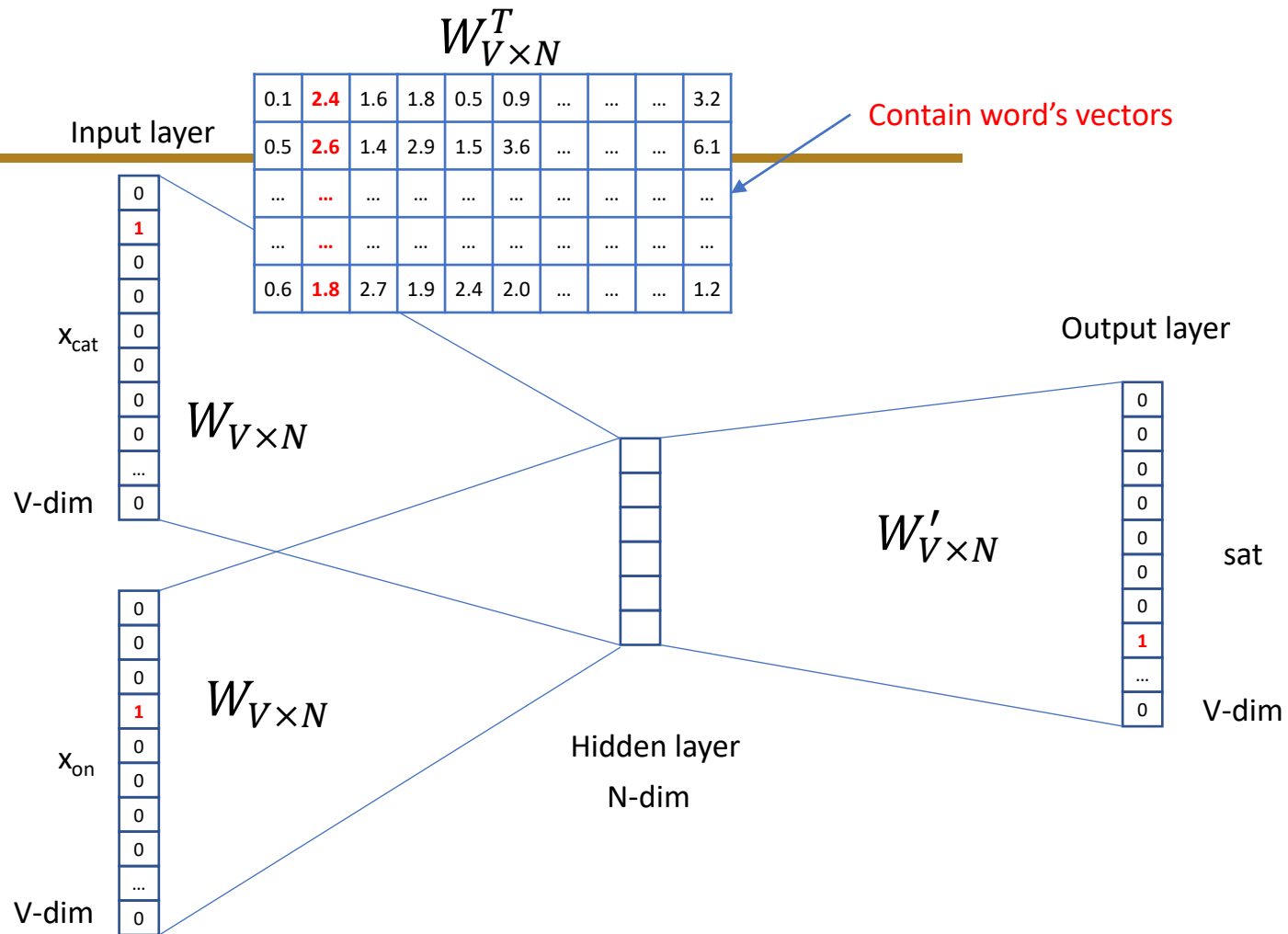












We can consider either W or W' as the word's representation. Or even take the average.

DANIYAL ADEEB

Some interesting results

— Word Analogies

Test for linear relationships, examined by Mikolov et al. (2014)

a:b :: c:?

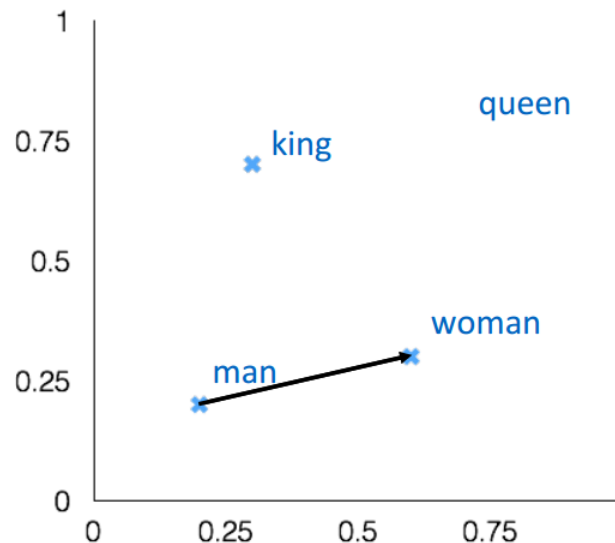
man:woman :: king:?

+ king [0.30 0.70]

- man [0.20 0.20]

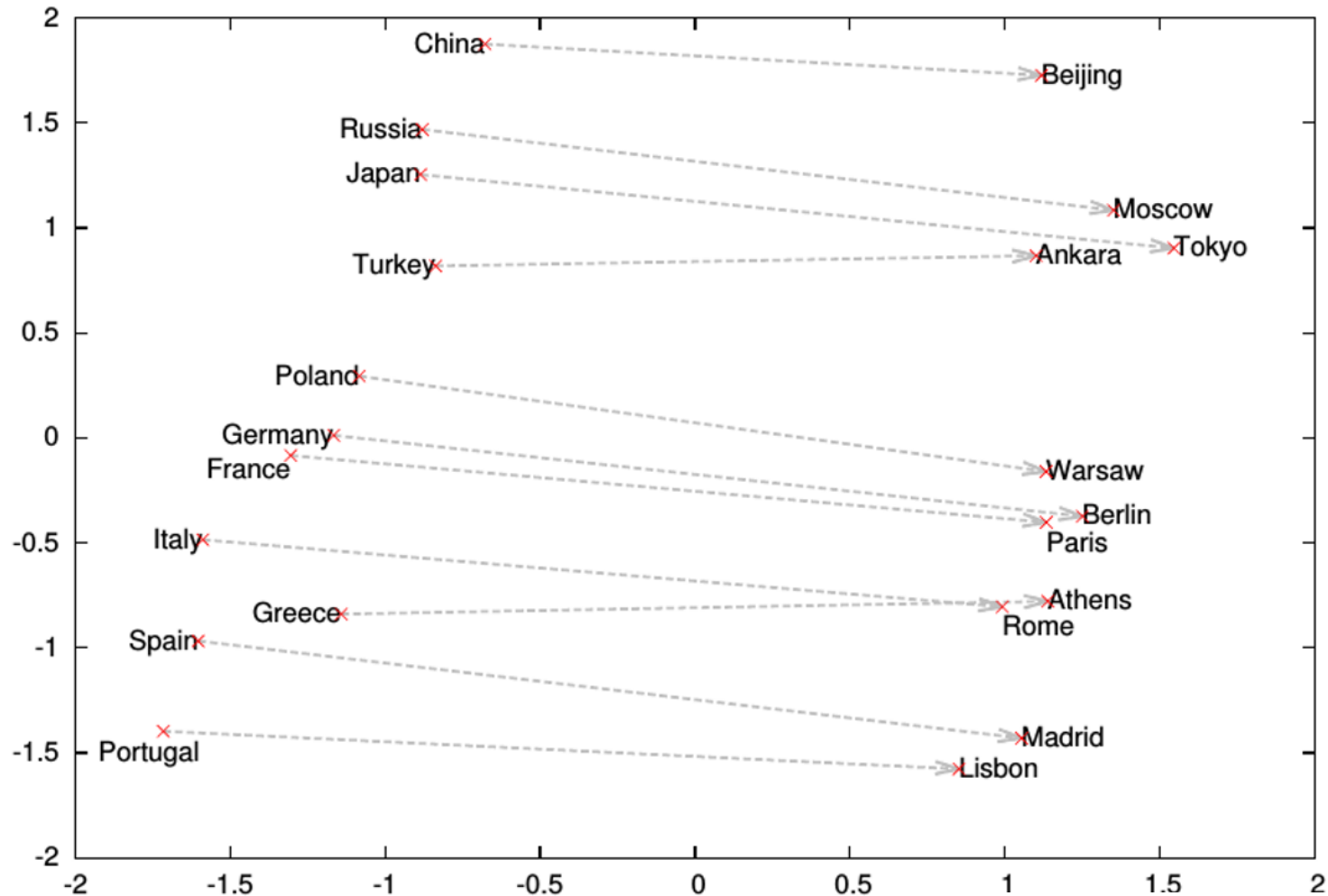
+ woman [0.60 0.30]

queen [0.70 0.80]



DANIYAL ADEEB

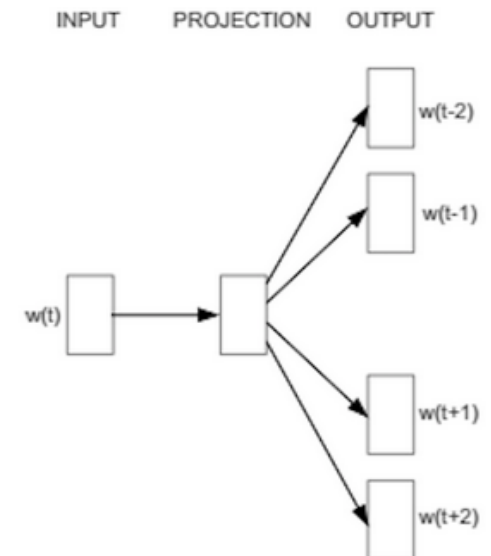
Word analogies



DANIYAL ADEEB

Skip gram

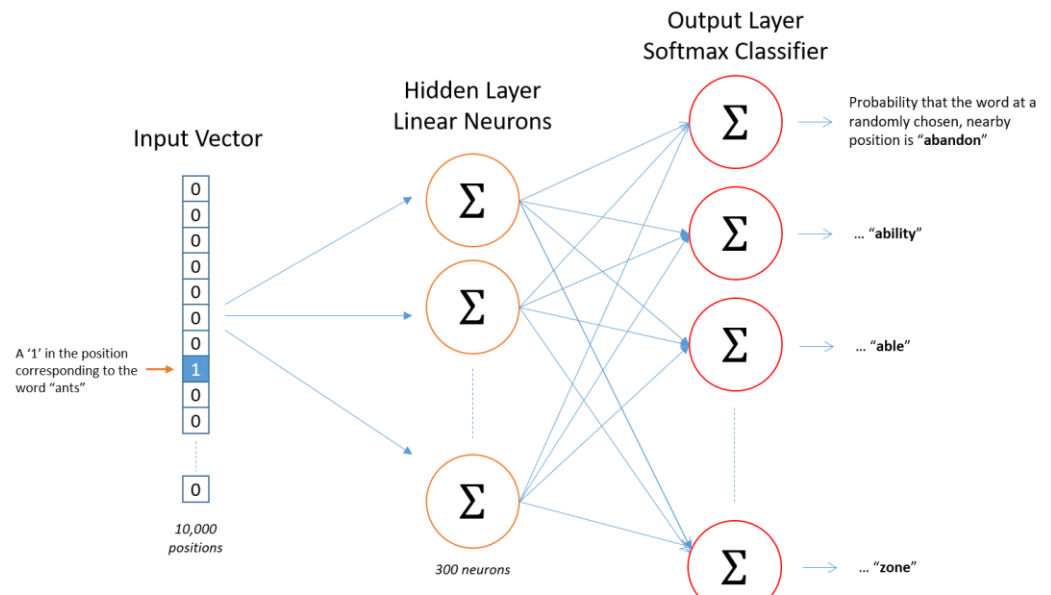
- Skip gram – alternative to CBOW
 - Start with a single word embedding and try to predict the surrounding words.
 - Much less well-defined problem, but works better in practice (scales better).



Skip-gram

Skip gram

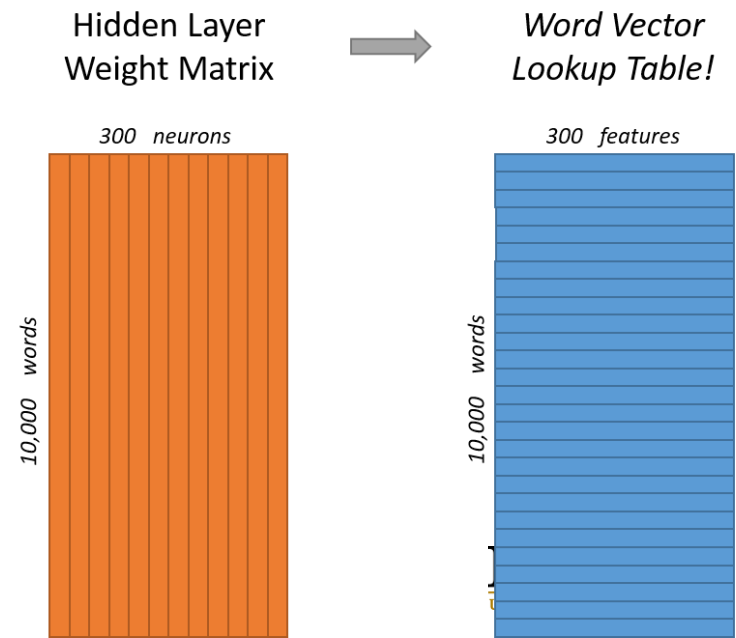
- Map from center word to probability on surrounding words. One input/output unit below.
 - There is no activation function on the hidden layer neurons, but the output neurons use softmax.



Skip gram example

- Vocabulary of 10,000 words.
- Embedding vectors with 300 features.
- So the hidden layer is going to be represented by a weight matrix with 10,000 rows (multiply by vector on the left).

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$



Skip gram/CBOW intuition

- Similar “contexts” (that is, what words are likely to appear around them), lead to similar embeddings for two words.
- One way for the network to output similar context predictions for these two words is if *the word vectors are similar*. So, if two words have similar contexts, then the network is motivated to learn similar word vectors for these two words!

Word2vec shortcomings

- **Problem:** 10,000 words and 300 dim embedding gives a large parameter space to learn. And 10K words is minimal for real applications.
- Slow to train, and need lots of data, particularly to learn uncommon words.

Word2vec improvements: word pairs and phrases

- **Idea:** Treat common word pairs or phrases as single “words.”
 - E.g., Boston Globe (newspaper) is different from Boston and Globe separately. Embed Boston Globe as a single word/phrase.
- **Method:** make phrases out of words which occur together often relative to the number of individual occurrences. Prefer phrases made of infrequent words in order to avoid making phrases out of common words like “and the” or “this is”.
- **Pros/cons:** Increases vocabulary size but decreases training expense.
- **Results:** Led to 3 million “words” trained on 100 billion words from a Google News dataset.

Word2vec improvements: subsample frequent words

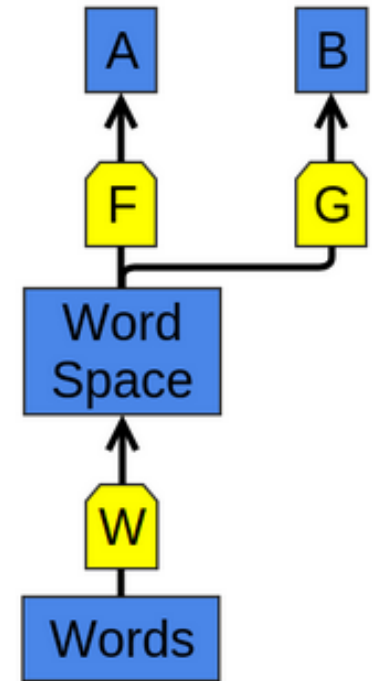
- **Idea:** Subsample frequent words to decrease the number of training examples.
 - The probability that we cut the word is related to the word's frequency. More common words are cut more.
 - Uncommon words (anything $< 0.26\%$ of total words) are kept
 - E.g., remove some occurrences of “the.”
- **Method:** For each word, cut the word with probability related to the word's frequency.
- **Benefits:** If we have a window size of 10, and we remove a specific instance of “the” from our text:
 - As we train on the remaining words, “the” will not appear in any of their context windows.

Word2vec improvements: selective updates

- **Idea:** Use “Negative Sampling”, which causes each training sample to update only a small percentage of the model’s weights.
- **Observation:** A “correct output” of the network is a one-hot vector. That is, one neuron should output a 1, and *all* of the other thousands of output neurons to output a 0.
- **Method:** With negative sampling, randomly select just a small number of “negative” words (let’s say 5) to update the weights for. (In this context, a “negative” word is one for which we want the network to output a 0 for). We will also still update the weights for our “positive” word.

Word embedding applications

- The use of word representations... has become a key “secret sauce” for the success of many NLP systems in recent years, across tasks including named entity recognition, part-of-speech tagging, parsing, and semantic role labeling. ([Luong et al. \(2013\)](#))
- Learning a good representation on a task A and then using it on a task B is one of the major tricks in the Deep Learning toolbox.
 - Pretraining, transfer learning, and multi-task learning.
 - Can allow the representation to learn from more than one kind of data.

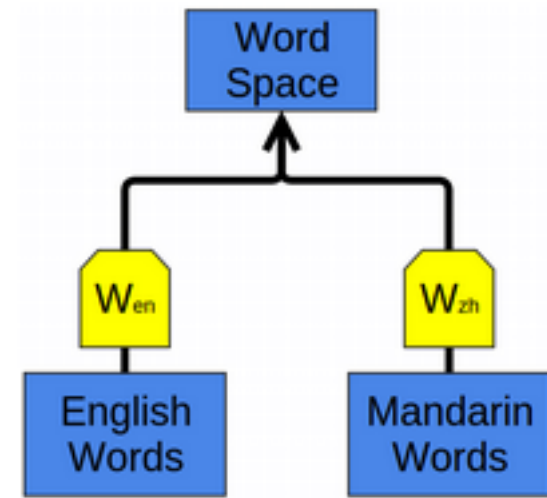


W and *F* learn to perform task A. Later, *G* can learn to perform B based on *W*.

DANIYAL ADEEB

Word embedding applications

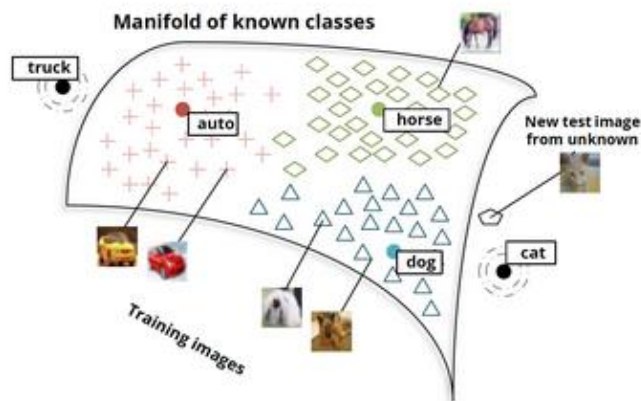
- Can learn to map multiple kinds of data into a single representation.
- E.g., bilingual English and Mandarin Chinese word-embedding as in [Socher et al. \(2013a\)](#).
- Embed as above, but words that are known as close translations should be close together.
- Words we *didn't know* were translations end up close together!
- Structures of two languages get pulled into alignment.



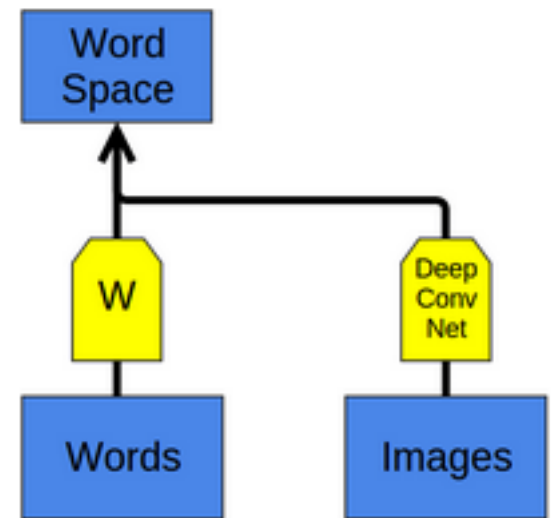
DANIYAL ADEEB

Word embedding applications

- Can apply to get a joint embedding of words and images or other multi-modal data sets.
- New classes map near similar existing classes: e.g., if 'cat' is unknown, cat images map near dog.



(Socher *et al.* (2013b))



DANIYAL ADEEB